

Complexity and Algorithms - Lecture 3

Efficient Binary Trees – Part 1

M.G.Noel A.S Fernando (PhD)

Professor

NSBM/Dept. of Information Systems Engineering, UCSC



Efficient Binary Trees

- The efficient binary trees such as [binary search trees](#), [AVL trees](#), [B-trees](#), [red-black trees](#), and [splay trees](#).

Binary Search trees

- A binary search tree, also known as an ordered binary tree, is a variant of binary trees in which the nodes are arranged in an order.
- In a binary search tree, all the nodes in the left sub-tree have a value less than that of the root node. Similarly, all the nodes in the right sub-tree have a value either equal to or greater than the root node.
- The same rule is applicable to every sub-tree in the tree.
- (Note that a binary search tree may or may not contain duplicate values, depending on its implementation.)

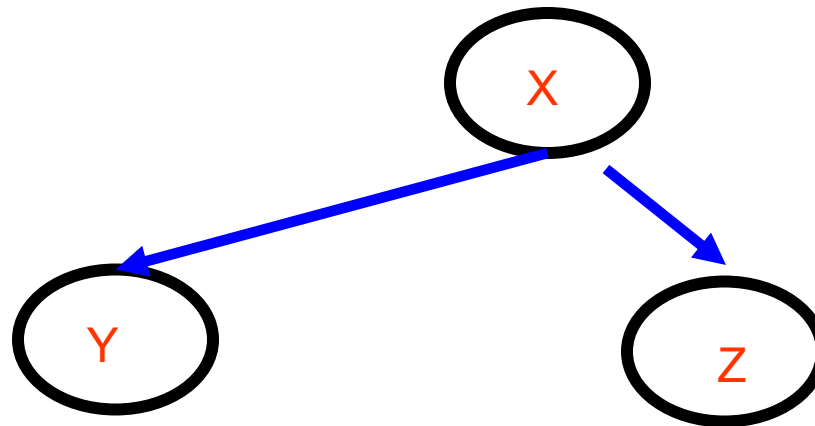
Binary Search Trees



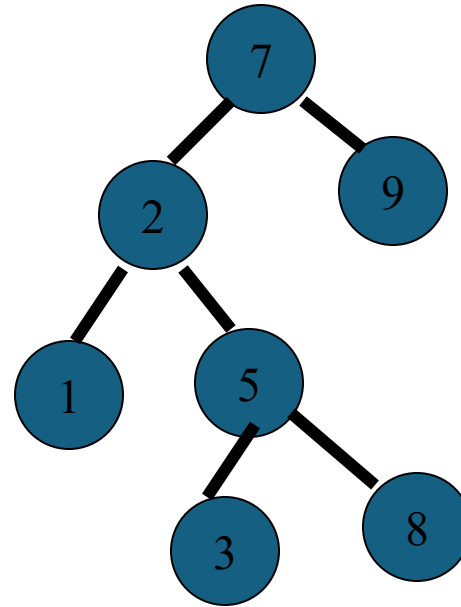
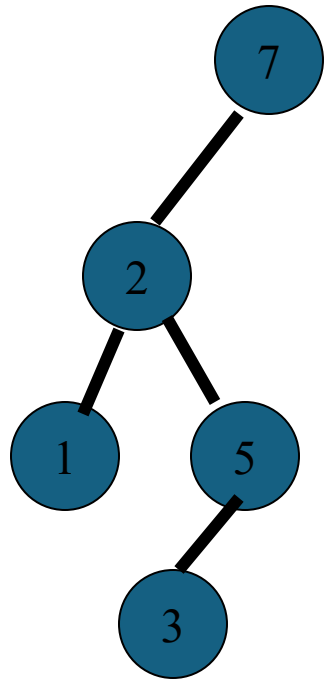
- In a binary search tree, each non-leaf node v stores an element e such that
 - ⌘ the elements stored in the left sub tree of v are less than e .
 - ⌘ the elements stored in the right sub tree of v are greater than e .

Binary Search Trees

- Key property
 - Value at node
 - Smaller values in left subtree
 - Larger values in right subtree
- Example
 - $X > Y$
 - $X < Z$

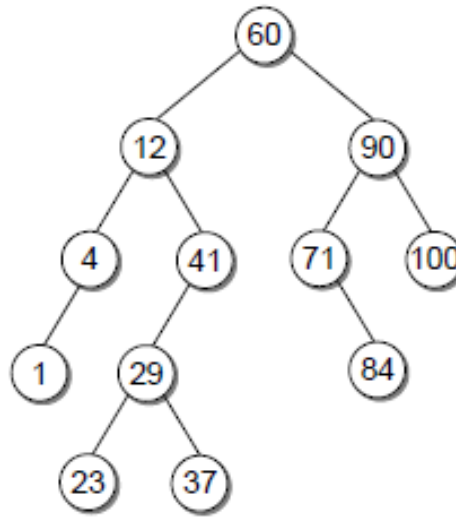


Examples :



Two binary trees(only the left tree is a search tree)

Binary Search Tree

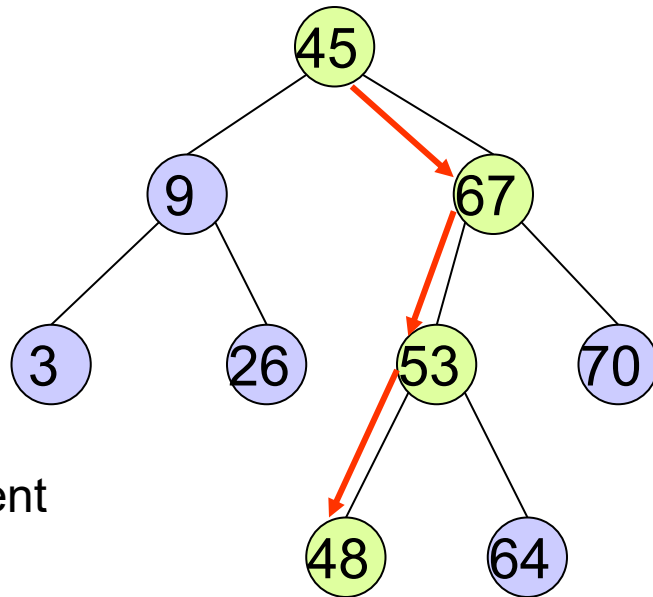


Given the relationship between the nodes, an in-order traversal will visit the nodes in increasing search key order.

For the example binary search tree, the order would be 1 4 12 23 29 37 41 60 71 84 90 100.

Binary Search Trees

- Advantage of using binary search trees —support fast search.



Find an element
in the binary
search tree

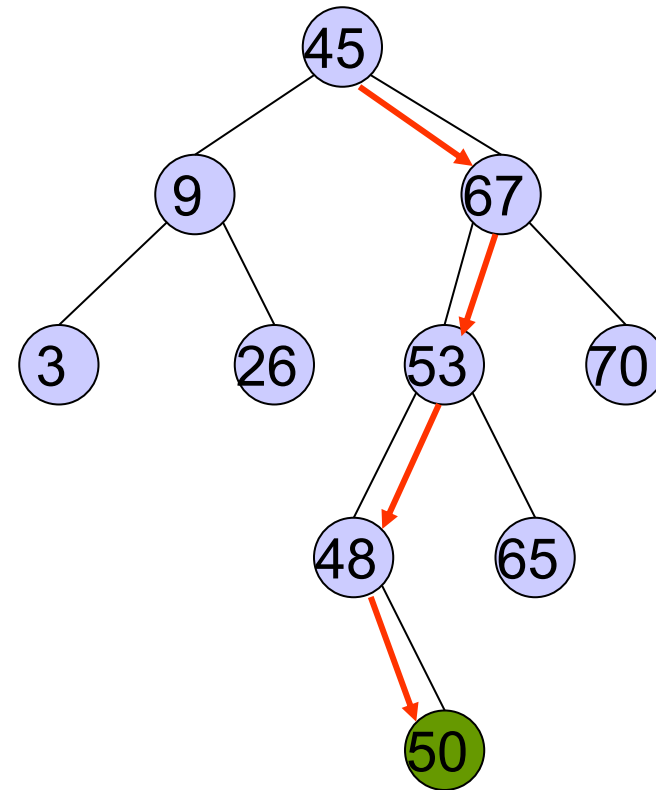
Binary Search Trees

- Adding a new element to a binary search tree.

☞ Example: add a new element 50 to the binary search tree.

☞ Ex. Create a binary search tree (words of dictionary) using the given key words

☞ Book, Sugar, River, Computer, Zoo, Jug, Fan, Pencil



Algorithm to search for a given value in a binary search tree

`searchElement (TREE, VAL)`

Step 1: IF `TREE → DATA = VAL` OR `TREE = NULL`

 Return `TREE`

ELSE

 IF `VAL < TREE → DATA`

 Return `searchElement(TREE → LEFT, VAL)`

 ELSE

 Return `searchElement(TREE → RIGHT, VAL)`

 [END OF IF]

 [END OF IF]

Step 2: END

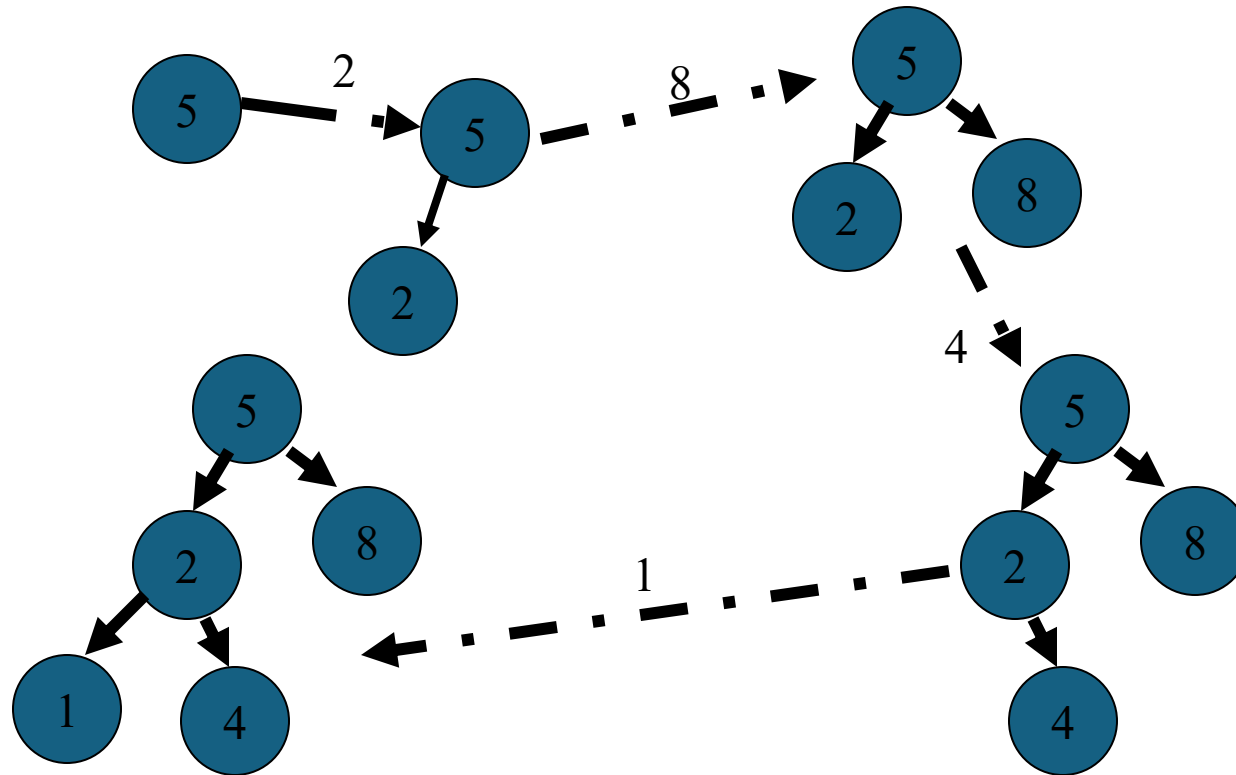
Applications : One way that a binary search tree can be used to sort data as follows.

- Eg: we have list of integers, such as 5,2,8,4, and 1 that we wish to sort in to ascending order.
- This algorithm consists of two stages.
- - Insertion
- - Traversal

Inserting- inserting integers into a binary search tree.

- **Step 1** : If the current pointer is null. Create a new node, store the data , and return the address of the new node.
- Step 2 : otherwise compare the integer to the data stored at the current, if the new node is less than the integer at the current node, insert the new integer into the left child of the current node(by recursively applying the same algorithm), otherwise, insert it into the right child of the current node.

Eg : 5,2,8,4 and 1



Algorithm to insert a given value in a binary search tree

Insert (TREE, VAL)

Step 1: IF TREE = NULL

 Allocate memory for TREE

 SET TREE → DATA = VAL

 SET TREE → LEFT = TREE → RIGHT = NULL

ELSE

 IF VAL < TREE → DATA

 Insert(TREE → LEFT, VAL)

 ELSE

 Insert(TREE → RIGHT, VAL)

 [END OF IF]

 [END OF IF]

Step 2: END

Note :

- The insert function requires time proportional to the height of the tree in the worst case.
- It takes $O(\log n)$ time to execute in the average case and $O(n)$ time in the worst case.

Binary Search Trees

- Adding a new element to a binary search tree.

Input: binary search tree T , node v , element e

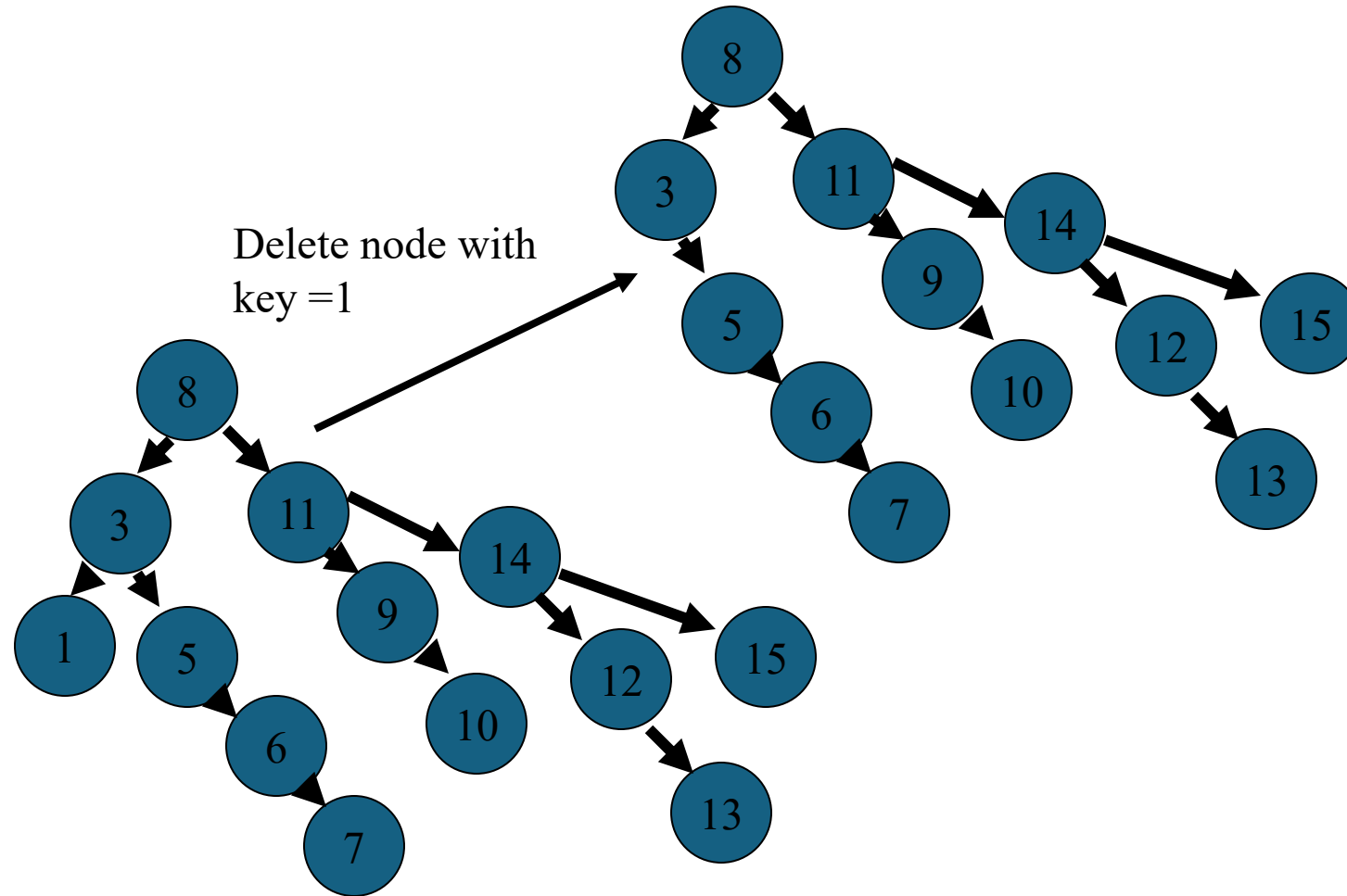
Output:

```
add(T, v, e){
    if(T.isLeaf(v)){
        if(v.element() >= e)
            add element e as v's left child
        else
            add element e as v's right child
    } else {
        if(v.element() >= e)
            add(T, T.leftChild(v), e)
        else
            add(T, T.rightChild(v), e)
    }
}
```

Deleting nodes from a binary search trees.

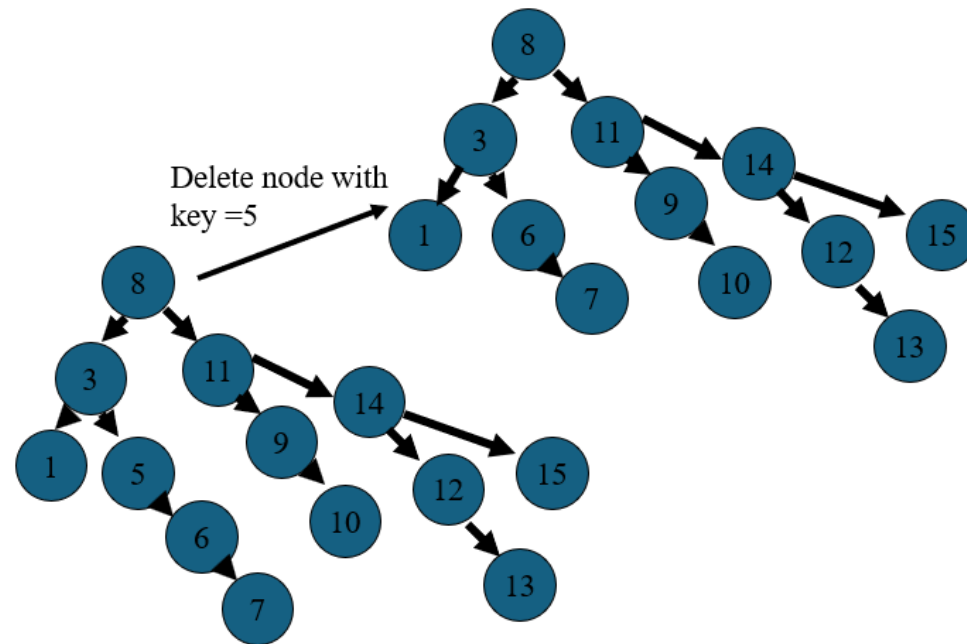
- There are three categories of nodes we may wish to delete.
- **Category 1** : The node to be deleted has no sons.
 - - The node can be deleted without any adjustment to the tree.
 - - Delete the leaf and set the pointer from its parent (if any) to zero.

Example :Category 1



Category 2 : The node to be deleted has only one sub-tree.

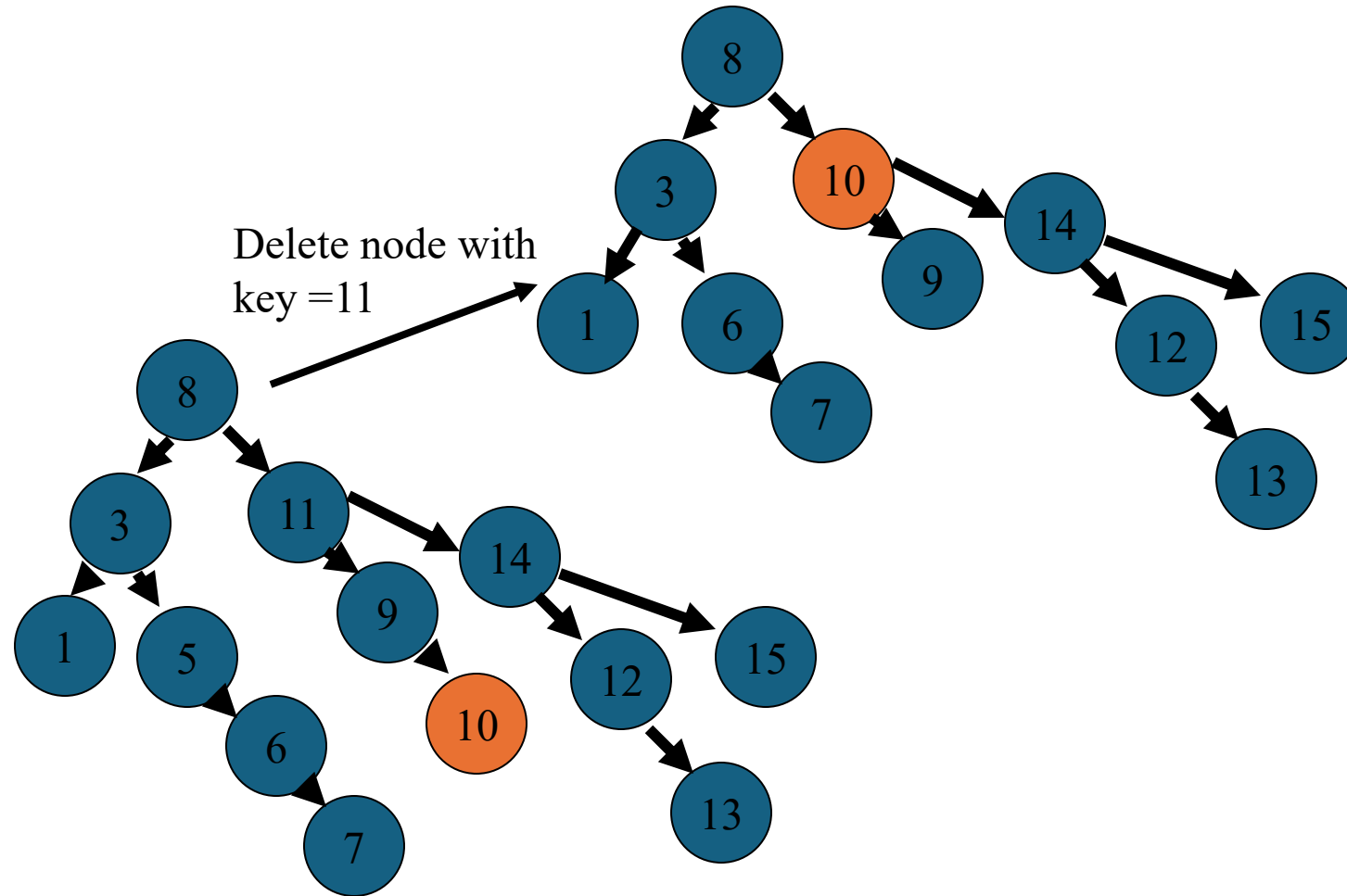
- Solution : Its only son can be moved up to its take place.
- We just redirect the references from the node's parent so that it points to the child.



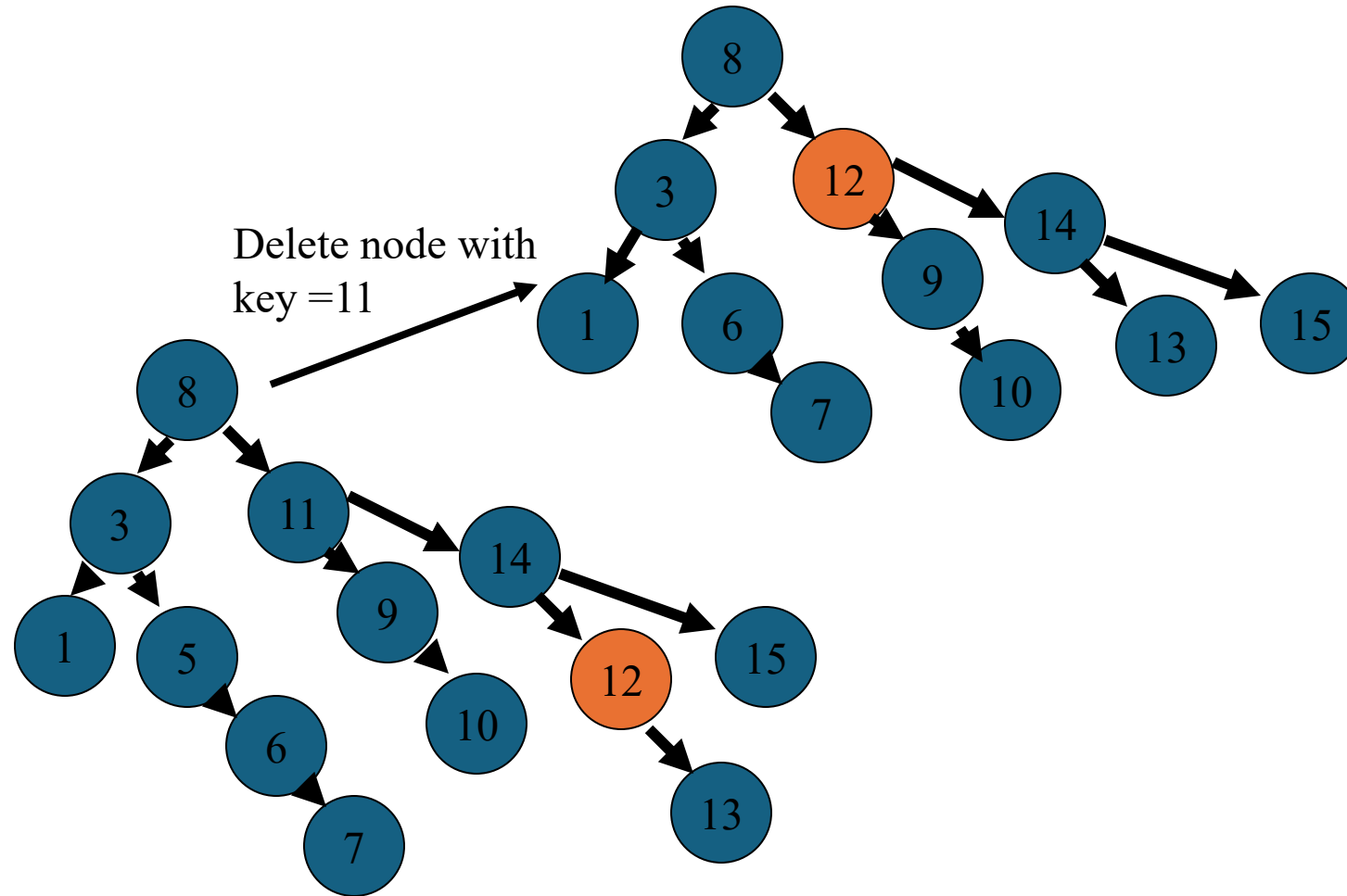
Category 3 : The node to be deleted has two sub-trees.

- Solution : this method we shall use is to replace the node being deleted by the rightmost node in its left sub-tree
- Or Leftmost node in its right-sub-tree.

Example :Category 3



Example :Category 3



Algorithm to delete a node from a binary search tree

Delete (TREE, VAL)

Step 1: IF TREE = NULL

 Write "VAL not found in the tree"

ELSE IF VAL < TREE->DATA

 Delete(TREE->LEFT, VAL)

ELSE IF VAL > TREE->DATA

 Delete(TREE->RIGHT, VAL)

ELSE IF TREE->LEFT AND TREE->RIGHT

 SET TEMP = findLargestNode(TREE->LEFT)

 SET TREE->DATA = TEMP->DATA

 Delete(TREE->LEFT, TEMP->DATA)

ELSE

 SET TEMP = TREE

 IF TREE->LEFT = NULL AND TREE->RIGHT = NULL

 SET TREE = NULL

 ELSE IF TREE->LEFT != NULL

 SET TREE = TREE->LEFT

 ELSE

 SET TREE = TREE->RIGHT

 [END OF IF]

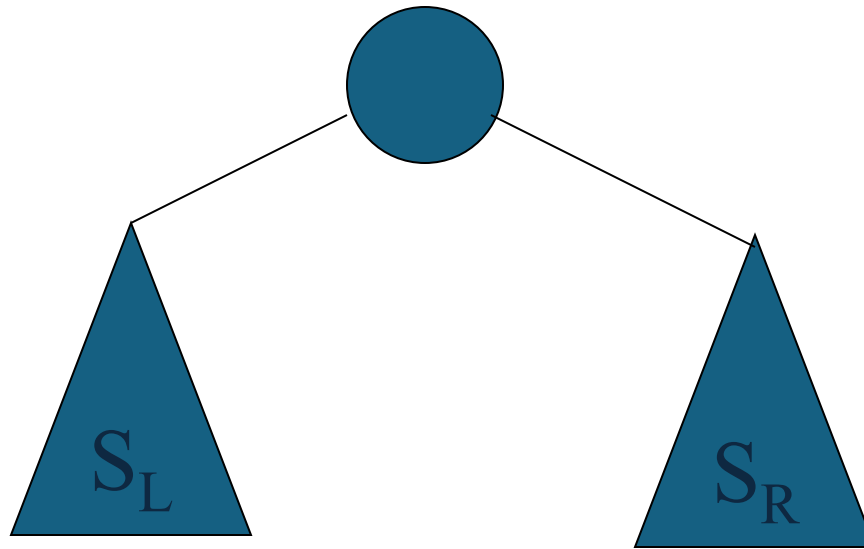
 FREE TEMP

 [END OF IF]

Step 2: END

Recursive view used to calculate the size of a tree : $S_T = S_L + S_R + 1$

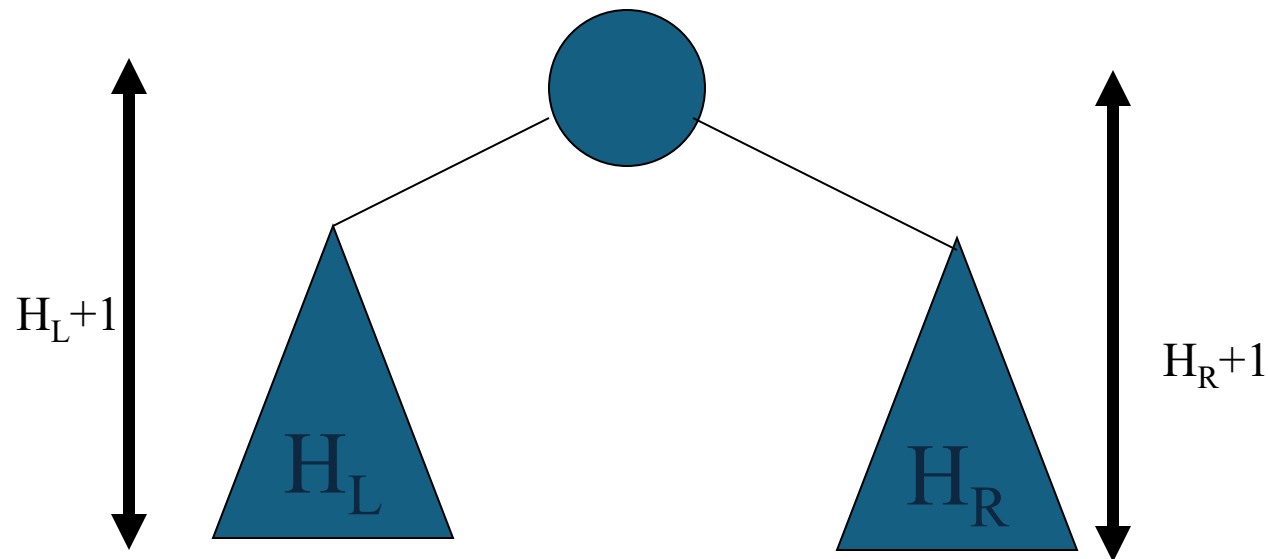
In order to determine the height of a binary search tree, we calculate the height of the left sub-tree and the right sub-tree.



Routine to compute the size of a node

```
/* Return the size of the binary tree rooted at t.  
if (t == null)  
    return 0  
else  
    return 1+size(t->left)+ size(t->right)  
}
```


Recursive view used to calculate the height of a tree : $H_T = \max(H_L + 1, H_R + 1)$



Routine to compute the height of a node

```
/* Return the height of the binary tree rooted at t.
```

```
if (t == null)
```

```
    return 0;
```

```
else
```

```
    return 1+maximum(height(t->left),height(t->right));
```

```
}
```

Other operations

- We can perform, find operation by starting at the root and then repeatedly branching either left or right, depending result of a comparison.

Findmin and findmax

- To perform a findmin, we start at root, and repeatedly branch left as long as there is a left child. The stopping point is the smallest element.
- Findmax routine is same, except that branching is to right child.

Findmin operation

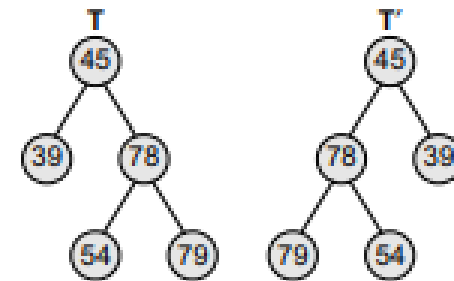
```
If (t == null)
return null;
else if (t->left == null)
return t;
return findMin(t->left);
}
```

FinMax(Non recursive)

```
{
    if (t!=null)
    While (t->right!=null)
    t=t->right;
    return t;
}
```

Finding the Mirror Image of a Binary Search Tree

- Mirror image of a binary search tree is obtained by interchanging the left sub-tree with the right sub-tree at every node of the tree.
- For example, given a tree T , the mirror image of T can be obtained as T_1 .
- Consider the following tree.



Finding the Mirror Image of a Binary Search Tree cont....

- A recursive algorithm more convenient to obtain the mirror image of a binary search tree.
- In the algorithm, if `TREE != NULL`, that is if the current node in the tree has one or more nodes, then the algorithm is recursively called at every node in the tree to swap the nodes in its left and right sub-trees.

Finding the Mirror Image of a Binary Search Tree cont....

`MirrorImage(TREE)`

`Step 1: IF TREE != NULL`

`MirrorImage(TREE -> LEFT)`

`MirrorImage(TREE -> RIGHT)`

`SET TEMP = TREE -> LEFT`

`SET TREE -> LEFT = TREE -> RIGHT`

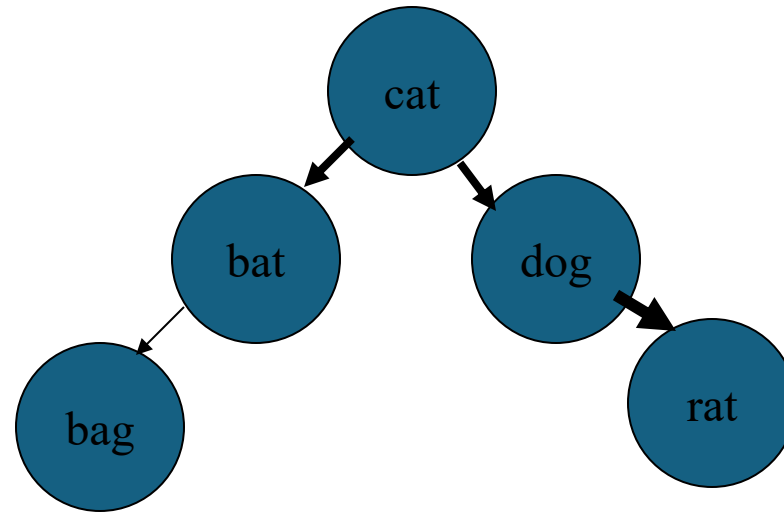
`SET TREE -> RIGHT = TEMP`

`[END OF IF]`

`Step 2: END`

Application : Implementation of a Dictionary [words]

- Eg : cat, bat, dog, rat, bag etc..



Homework

- Write a program to create a binary search tree and perform all the operations discussed in the preceding sections.